# bhttp/1: HTTP in binary frames

Version 1 · Raj Prakash · HTTP semantics in length-prefixed binary frames over one persistent TCP connection.

## 1. Conventions

MUST, MUST NOT, SHOULD, SHOULD NOT and MAY are as in RFC 2119. Integers are unsigned and **big-endian** (network byte order); widths are in bits. A *message* is a request or a response.

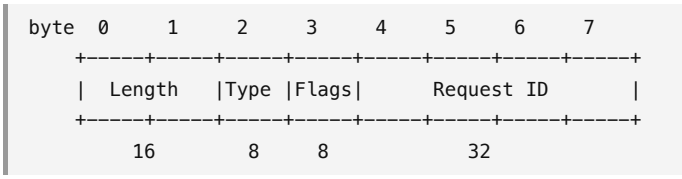
## 2. Connection

bhttp/1 runs over TCP with no registered port (examples use 9000) and no preface: the client's first bytes are its first frame.

- **Persistent and sequential.** A client MUST NOT send a REQUEST before the previous response's END has arrived; a server MUST answer requests in the order received.
- **The client closes** when it is done (the normal end); it MAY send GOAWAY first.
- **The server closes** (a) after a connection error (§7); (b) when no complete REQUEST arrives within T seconds of accepting the connection or finishing the previous response (in a request body: no DATA within T of its previous frame); other and partial frames do not extend T, which SHOULD be ≥ 10 s; (c) to refuse a connection it cannot take, with GOAWAY Last-ID 0, Code `0x02` BUSY (the client MAY retry later).
- A server closing on its own initiative SHOULD send GOAWAY, shut down its sending side, and read and discard input for up to 2 s, so the peer reads the GOAWAY, not a reset.
- A receiver whose connection ends mid-frame discards the partial frame and closes.

## 3. Frame header

Every frame is an 8-byte header followed by Length bytes of payload.




- **Length (16):** payload bytes after the header. MUST be ≤ **16384**, whatever the Type; a larger value is a connection error (§7).
- **Type (8):** §4. **Flags (8):** `0x01` = **END**; other bits MUST be 0 when sent and MUST be ignored on receipt.
- **Request ID (32):** a client MUST number its requests 1, 2, 3, … on each connection, each exactly one more than the last, and after 0xffffffff MUST open a new connection. The server copies the ID into every frame of the response and checks only that it is not 0. ID 0 names the connection, as stream 0 does in HTTP/2: a REQUEST, RESPONSE or DATA with ID 0 sent to a server is a connection error (§7); a RESPONSE with ID 0 reports one.

**Why these widths.** HTTP/2 (RFC 9113) uses 24 / 8 / 8 / 1+31 bits (9 bytes). Its reasons, and bhttp/1's:

| Field | HTTP/2 chose … because | bhttp/1 chooses … because |
|---|---|---|
| Length | **24.** `SETTINGS_MAX_FRAME_SIZE` is 2^14 by default and may be raised to 2^24 − 1. The default is small because streams share the connection: one huge DATA frame would stall every other stream behind it (head-of-line blocking). | **16,** capped at the same 2^14; with no SETTINGS, a wider field would only describe frames nobody may send. The 16th bit lets a later version negotiate 64 KiB; a small cap keeps a multiplexed v2 fair, as in HTTP/2. Text such as `GET /` reads as Length ≥ 16640 and fails at once. |
| Type | **8:** 256 types; unknown types MUST be ignored and discarded, which is how HTTP/2 is extended. | **8,** with the same rule (§4), which leaves room for version 2. v1 uses 4 types. |
| Flags | **8** per-type booleans (END_STREAM, END_HEADERS, PADDED, PRIORITY, ACK): a bit means different things in different types. | **8.** v1 needs one bit, END, with one meaning; seven stay free. |
| ID | **1 + 31.** R MUST be 0 when sent and is ignored on receipt. Client streams are odd and server-push streams even, and IDs are never reused: a client gets 2^30 streams, then must open a new connection. Stream 0 is the connection. | **32,** no R bit or odd/even split: with no push or multiplexing, order alone matches responses. The ID stays to catch a desynchronised connection, and because the header can never change, so room for multiplexing is reserved now. Fields stay aligned. |

## 4. Frame types

| Type | Name | From | ID | Payload |
|---|---|---|---|---|
| `0x01` | REQUEST | client | new | Method, Path, headers |
| `0x02` | RESPONSE | server | copied | Status, headers |
| `0x03` | DATA | both | same | 0–16384 body bytes |
| `0x04` | GOAWAY | both | 0 | Last-ID (32), Code (8) |

**END** on a REQUEST or RESPONSE means "no body follows"; on DATA, "last frame of the body"; on GOAWAY it is ignored.

**Unknown types: the MUST-skip rule.** A receiver that meets a frame whose Type it does not know MUST read and discard exactly Length payload bytes and carry on as if the frame were absent, whatever its Flags and ID and wherever it appears, also between the frames of a message. It MUST NOT reply to it or close because of it; only the Length limit applies. Types `0x00` and `0x05–0x7f` are reserved for later versions of this spec and `0x80–0xff` for private experiments; a v1 sender MUST NOT send them.

**GOAWAY** means "I am closing; send no more requests". Its ID is 0 and is ignored on receipt. *Last-ID*: from a server, the last request it answered completely (0 if none); a client sends 0. *Code*: `0x00` NO_ERROR (idle, shutdown), `0x01` PROTOCOL_ERROR, `0x02` BUSY (§2); other values count as `0x01`. Bytes after the first five are ignored; a shorter payload reads as Last-ID 0, Code `0x01`. The receiver MUST NOT send another REQUEST and SHOULD close; requests above Last-ID were not processed and MAY be retried elsewhere.

## 5. Payloads

**5.1 REQUEST:** `Method (8) | Path Length (16) | Path | header block`. Methods: `0x01` GET, `0x02` HEAD, `0x03` POST, `0x04` PUT, `0x05` DELETE, as in HTTP; any other code is mal-

formed. The Path is 1 to Length − 3 raw bytes, never percent-decoded; everything from its first `?` is the query. It is malformed if it contains a byte `0x00–0x1f` or `0x7f`, or if the part before the query does not start with `/` or has a segment `.` or `..`.

**5.2 RESPONSE:** `Status (16) | header block`, an HTTP status from 200 to 599.

**5.3 Header block.** Entries back to back, filling the rest of the payload exactly:

```
indexed | Index 1..10 | VLen (16) | Value |
literal | 0x00 | NLen (8) | Name | VLen (16) | Value |
```

| Index | Name | Index | Name |
|---|---|---|---|
| 1 | `host` | 6 | `last-modified` |
| 2 | `user-agent` | 7 | `date` |
| 3 | `accept` | 8 | `server` |
| 4 | `content-type` | 9 | `location` |
| 5 | `content-length` | 10 | `allow` |

These are exactly the ten names bcurl (1–3) and bserve (4–10) send. A table name SHOULD be sent indexed; receivers MUST accept the literal form too. An entry is malformed if its Index is 11–255, if it runs past the payload, if a literal Name is empty or not lowercase `a–z 0–9 ! # $ % & ' * + - . ^ _ | ~` and backtick, or if a Value has `0x00`, `0x0a` or `0x0d`. Repeated names have HTTP meaning; dates are IMF-fixdate. Clients SHOULD send `host`; servers MUST NOT require any header.

**Payload widths.** A byte suffices for Method, Index and NLen (5 methods, 10 names, names ≤ 255 bytes); Path Length, VLen and Status take 16 bits (long values; 599 > 255). As in HPACK the static table is 1-based and Index 0 means "a literal name follows". Unlike HPACK, integers are fixed-width (no prefix-varints) and strings are not Huffman-coded: both save bytes but need a more complex decoder. Nor is there a dynamic table, the one HPACK part that makes a decoder stateful.

## 6. Messages and bodies

A message is a REQUEST or RESPONSE frame and, if it lacks END, a body of DATA frames with the same ID, the last carrying END. Only a REQUEST opens a body at a server. Senders SHOULD fill every DATA frame but the last; an empty one is allowed (useful with END).

`content-length` is optional: 1 to 19 ASCII digits (leading zeros allowed); any further copy MUST be byte-for-byte identical (`3` and `03` are not); it MUST equal the body length, except that every response to a REQUEST whose Method byte is HEAD, whatever its status, sets END on the RESPONSE and has no DATA (a 200 carries the headers a GET would get). A receiver treats a violation as malformed.

A server MUST read a request body through its END before it treats a further frame as a new request. A server that fails after sending RESPONSE MUST close without sending END.

## 7. Errors

**Connection errors** break the framing; the stream cannot be resynchronised: a Length above 16384 (the frame is not read further); a REQUEST, RESPONSE or DATA with ID 0; and, while a request body is open, any *known* frame type other than DATA for that ID (unknown types are still skipped, §4; a GOAWAY just ends the connection). The server sends RESPONSE **400** with ID 0 (optionally with a body), then GOAWAY `0x01`, and closes.

**Request errors** leave the framing intact: the next frame starts where expected. The server answers RESPONSE **400** with the offending frame's ID and **keeps the connection open**. They are a REQUEST payload under 4 bytes, an unknown method, a Path Length of 0 or past the payload, a Path rule violation (§5.1), a malformed header block, a bad `content-length` (§6), a RESPONSE sent to a server (it opens no body), and a DATA frame with no open request body. A malformed REQUEST without END still has its body read first.

**Other replies: 404** not found, outside the root, hidden, or not a regular file; **403** not readable; **301** with `location` = the path without its query, plus `/`, for a directory named without a trailing `/`; **405** with `allow` for a method the server does not implement; **500** for a read error before RESPONSE.

**Clients** close on a protocol error: Length > 16384, a RESPONSE or DATA with a wrong ID (ID 0 reports a connection error), DATA before the RESPONSE, a second RESPONSE, a REQUEST, a malformed payload, a status outside 200–599, a bad `content-length`, or EOF before END. A GOAWAY before END means the request was not processed (it MAY be retried).

## 8. Path mapping

A server serving the directory ROOT drops the query; appends `index.html` if the path ends in `/`; answers 404 if a segment starts with `.`; resolves ROOT + path following symbolic links and answers 404 if the result is outside ROOT (resolved the same way); answers 301 for a directory, or 404 if the path already ended in `/`; and 404 for anything else that is not a regular file. This MUST hold for what is actually opened even if the tree changes meanwhile (say, open the resolved name component by component without following links). `content-type` SHOULD follow the file extension.

## 9. Versions

No version number is sent; the frame header and the MUST-skip rule are frozen. A later version may add frame types (`0x05–0x7f`), flag bits, GOAWAY codes and bytes, and frames up to Length 65535, but uses them only once the peer has shown it understands them: a v2 peer announces itself with a new frame type (say, HELLO) that a v1 peer skips, so a v2 client that sees a RESPONSE first stays at v1.

## 10. Worked example

GET `/` with no headers, ID 1, answered by a 404 with no body (HEXDUMP.md annotates a full exchange):

```
00 04 01 01 00 00 00 01   Length 4, REQUEST, END, ID 1
01 00 01 2f               GET, Path Length 1, "/"
00 02 02 01 00 00 00 01   Length 2, RESPONSE, END, ID 1
01 94                     Status 0x0194 = 404
```

One connection, frame by frame:

```
C>S  REQUEST  id=1 END   GET /a.bin
S>C  RESPONSE id=1       200, content-length: 20000
S>C  DATA     id=1       16384 bytes
S>C  DATA     id=1 END   3616 bytes
C>S  type 0x42 id=0      unknown: skipped, no reply
C>S  REQUEST  id=2 END   GET /x/../y
S>C  RESPONSE id=2 END   400: request error, kept open
       ... T seconds without a REQUEST ...
S>C  GOAWAY   id=0       Last-ID 2, NO_ERROR; S closes
```

## 11. Reference client (informative)

bcurl fetches `[bhttp://]host[:port][path]` URLs (host may be `[IPv6]`; port 9000) in order over one connection, never reopened. Bodies go to stdout; `-v` hexdumps every frame to stderr. It gives up after `-t` seconds (30) without a RESPONSE or DATA. Exit: 0 all 2xx/3xx; 4 or 5 worst 4xx or 5xx; 2 connection or protocol error or timeout; 1 usage error (bad URL or option, or a REQUEST too big for one frame).